Follow-on locking

The Oracle `Dialect` cannot employ the `FOR UPDATE` clause when the underlying query uses pagination because otherwise the database throws the following exception:

```
ORA-02014: cannot select FOR UPDATE from view with DISTINCT, GROUP BY, etc.
```

Because the original query cannot use the `FOR UPDATE` clause, each matching row must be locked with a secondary select statement.

After Alice has locked some `Post` records and started to moderate them, Bob decides to do the same thing, but, when he tries to run the same query as Alice, he will get an exception because the same rows are already locked. To address this usability issue, some database systems (e.g. Oracle 10g, PostgreSQL 9.5) define a `SKIP LOCKED` clause so a query can filter out row entries that are already locked. The following example is going to demonstrate how `SKIP LOCKED` works:

```
private List<Post> pendingPosts(EntityManager entityManager, int lockCount,
                                int maxResults, Integer maxCount) {
    LOGGER.debug("Attempting to lock {} Post(s) entities", maxResults);
    List<Post> posts= entityManager.createQuery(
        "select p from Post p where p.status = :status", Post.class)
    .setParameter("status", PostStatus.PENDING)
    .setMaxResults(maxResults)
    .unwrap(org.hibernate.Query.class)
    .setLockOptions(new LockOptions(LockMode.UPGRADE_SKIPLOCKED))
    .list();

    if(posts.isEmpty()) {
        if(maxCount == null) {
            maxCount = pendingPostCount(entityManager);
        }
        if(maxResults < maxCount || maxResults == lockCount) {
            maxResults += lockCount;
            return pendingPosts(entityManager, lockCount, maxResults, maxCount);
        }
    }
    LOGGER.debug("{} Post(s) entities have been locked", posts.size());
    return posts;
}
```

The `pendingPostCount` method calculates the maximum number of `Post` entities that are eligible for moderation.

```
private int pendingPostCount(EntityManager entityManager) {
    int postCount = ((Number) entityManager.createQuery(
        "select count(*) from Post where status = :status")
    .setParameter("status", PostStatus.PENDING)
    .getSingleResult()).intValue();

    LOGGER.debug("There are {} PENDING Post(s)", postCount);
    return postCount;
}
```

Because the aforementioned `pendingPost` is private, the following simplified overloaded method is going to be used by the service layer:

```
public List<Post> pendingPosts(EntityManager entityManager, int lockCount) {
    return pendingPosts(entityManager, lockCount, lockCount, null);
}
```

With this new method in place, Alice and Bob can moderate distinct `Post` entries without risking any pessimistic locking conflict.

```
doInJPA(entityManager -> {
    final int lockCount = 2;
    LOGGER.debug("Alice wants to moderate {} Post(s)", lockCount);
    List<Post> pendingPosts = pendingPosts(entityManager, lockCount);
    List<Long> ids = pendingPosts
        .stream().map(Post::getId).collect(toList());
    assertTrue(ids.size() == 2 && ids.contains(0L) &&
        ids.contains(1L));

    executeSync(() -> {
        doInJPA(_entityManager -> {
            LOGGER.debug("Bob wants to moderate {} Post(s)", lockCount);
            List<Post> _pendingPosts = pendingPosts(_entityManager, lockCount);
            List<Long> _ids = _pendingPosts
                .stream().map(Post::getId).collect(toList());
            assertTrue(_ids.size() == 2 &&
                _ids.contains(2L) && _ids.contains(3L));
        });
    });
});
```

When running the aforementioned test case, Hibernate generates the following output:

```
-- Alice wants to moderate 2 Post(s)
-- Attempting to lock 2 Post(s) entities
SELECT * FROM (
    SELECT p.id AS id1_0_, p.body AS body2_0_, p.status AS status3_0_,
           p.title AS title4_0_, p.version AS version5_0_
    FROM   post p
    WHERE  p.status = 0
)
WHERE  rownum <= 2
FOR UPDATE SKIP LOCKED
-- 2 Post(s) entities have been locked

-- Bob wants to moderate 2 Post(s)
-- Attempting to lock 2 Post(s) entities
SELECT * FROM (
    SELECT p.id AS id1_0_, p.body AS body2_0_, p.status AS status3_0_,
           p.title AS title4_0_, p.version AS version5_0_
    FROM   post p
    WHERE  p.status = 0
)
WHERE  rownum <= 2
FOR UPDATE SKIP LOCKED

SELECT COUNT(*) AS col_0_0_
FROM   post p
WHERE  p.status = 0
-- There are 10 PENDING Post(s)

-- Attempting to lock 4 Post(s) entities
SELECT * FROM (
    SELECT p.id AS id1_0_, p.body AS body2_0_, p.status AS status3_0_,
           p.title AS title4_0_, p.version AS version5_0_
    FROM   post p
    WHERE  p.status = 0
)
WHERE  rownum <= 4
FOR UPDATE SKIP LOCKED
-- 2 Post(s) entities have been locked
```

The flow can be explained as follows:

- The `lockCount` variable dictates how many `Post` entities a user should be locking at once.
- Alice tries to lock 2 `Post` entities with a status of `PENDING`, and since no other user has locked any such entity, she manages to lock the first 2 `Post` records.
- Bob also attempts to lock 2 `Post` entities.
- At first, Bob's tries to lock 2 `PENDING` `Post(s)`, but the query returns no record. This happens because the `SKIP LOCKED` clause ignores the matching records that are already locked (by Alice).
- Bob counts the number of `Post` entities to know many records are eligible for moderation. Even if Alice locked two rows, because Oracle uses MVCC, the `pendingPostCount` query is able to count both locked and unlocked database table records.
- Knowing that there are still some records that might not be locked, he increments the `maxResults` variable with the `lockCount` value.
- The `maxResults` tells the maximum number of entities that can be scanned by the current iteration.
- Because the `maxResults` has a value of 4, there are 4 `Post` records being scanned. However, since Alice has locked the first two entries (identifiers 0 and 1), Bob can only lock the next 2 records (identifiers 2 and 3).
- Because Bob has managed to lock at least one `Post` entity, he can continue with the moderation process.

LockMode.UPGRADE_SKIPLOCKED

Long before JPA 1.0, Hibernate defined its own `LockMode(s)`, which have been later used as the base of the Java Persistence `LockModeType(s)`. Although JPA 2.1 does not offer support for skipping locks, when using Hibernate, by setting the timeout value to `LockOptions.SKIP_LOCKED` (e.g. value of -2) the `SKIP LOCKED` clause is applied to the pessimistic locking clause. However, because of the follow-on locking behavior on Oracle, the `SKIP LOCKED` cannot by applied to the original query, so, not only the expected goal is not achieved, but this query will fail due to a stale state false positive. If a given row is already locked, the secondary follow-on locking query will not find any row, and Hibernate is going to assume that the row version has changed, or the row was deleted in the meanwhile, causing an `OptimisticLockingException`.

Fortunately, with Hibernate 5.1, the `LockMode.UPGRADE_SKIPLOCKED` bypasses the follow-on locking mechanism, as demonstrated by the previous example. Nevertheless, the locking query cannot use any `ORDER BY`, `GROUP BY`, or offset pagination. Otherwise, Oracle is going to throw an exception[a].

[a] https://docs.oracle.com/database/121/SQLRF/statements_10002.htm#SQLRF55371

Hibernate 5.2.1 follow-on locking improvements

Since Hibernate 5.2.1, the Oracle Dialect does not resort to *follow-on locking* on every situation. Therefore, the follow-on locking mechanism is activated if the underlying query contains one of the subsequent directives:

- `DISTINCT`
- `GROUP BY`
- `UNION` or `UNION ALL`
- Pagination with `ORDER BY` or with OFFSET (e.g. `setFirstResult`)

For this reason, on Hibernate 5.2.1, the previous example which was using `UPGRADE_SKIPLOCKED` `LockMode` to bypass the follow-on locking mechanism can be rewritten as follows:

```
List<Post> posts= entityManager.createQuery(
    "select p from Post p where p.status = :status", Post.class)
.setParameter("status", PostStatus.PENDING)
.setMaxResults(maxResults)
.unwrap(org.hibernate.Query.class)
.setLockOptions(new LockOptions(LockMode.PESSIMISTIC_WRITE)
    .setTimeOut(LockOptions.SKIP_LOCKED))
.list();
```

The aforementioned query works since it does not use any directive that would otherwise require the follow-on locking mechanism.

More, if there is any situation where the follow-on locking mechanism is being chosen although the underlying SQL query can successfully apply the row-level lock acquisition request, the `LockOptions` now offers the possibility of manually setting the follow-on locking strategy:

```
List<Post> posts= entityManager.createQuery(
    "select p from Post p where p.status = :status", Post.class)
.setParameter("status", PostStatus.PENDING)
.setMaxResults(maxResults)
.unwrap(org.hibernate.Query.class)
.setLockOptions(new LockOptions(LockMode.PESSIMISTIC_WRITE)
    .setTimeOut(LockOptions.SKIP_LOCKED)
    .setFollowOnLocking(false))
.list();
```

16.2.2 LockModeType.OPTIMISTIC

To understand how `LockModeType.OPTIMISTIC` works, the following entities are going to be used in the upcoming test cases:

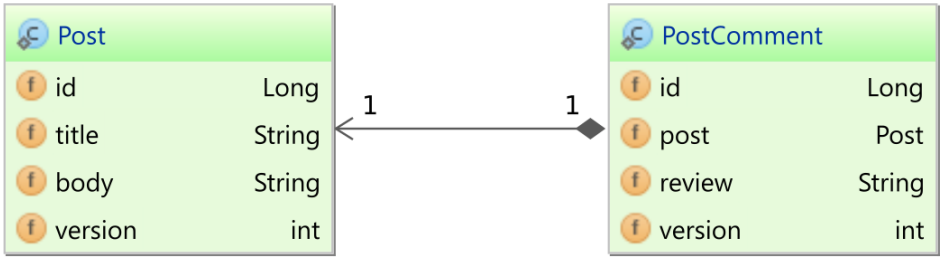

Figure 16.8: Post and PostComment entities

Once a `Post` is published, users can add `PostComment(s)` to review and share their opinions about the content of the aforementioned `Post`. Even if both entities have a `version` attribute, lost updates can be prevented at the entity level only. However, the `PostComment` entity is strictly related to the state of the `Post` entity that was used for reviewing. If a concurrent user modified the `Post` entity content, the `PostComment` might no longer be relevant.

In the following example, Alice is going to select a `Post` entity, and, while she is reviewing the `Post` entity, Bob is changing its content so that it now references the 17th chapter of the book. Alice, being unaware of the latest `Post` change, she adds a `PostComment` for the 16th chapter of the book.

```
doInJPA(entityManager -> {
    LOGGER.info("Alice loads the Post entity");
    Post post = entityManager.find(Post.class, 1L);

    executeSync(() -> {
        doInJPA(_entityManager -> {
            LOGGER.info("Bob loads the Post entity and modifies it");
            Post _post = _entityManager.find(Post.class, 1L);
            _post.setBody("Chapter 17 summary");
        });
    });

    LOGGER.info("Alice adds a PostComment to the previous Post entity version");
    PostComment comment = new PostComment();
    comment.setId(1L);
    comment.setReview("Chapter 16 is about Caching.");
    comment.setPost(post);
    entityManager.persist(comment);
});
```

When executing the test case above, Hibernate generates the following output:

```sql
-- Alice loads the Post entity
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

-- Bob loads the Post entity and modifies it
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

UPDATE post
SET    body = 'Chapter 17 summary' ,
       title = 'High-Performance Java Persistence' ,
       version = 1
WHERE  id = 1 AND version = 0

-- Alice adds a PostComment review to the previous Post entity version
INSERT INTO post_comment (post_id, review, version, id)
VALUES (1, 'Chapter 16 is about Caching.', 0, 1)
```

This is still a lost update that would never happen if Alice were taking a shared lock on the `Post` entity. Unfortunately, a shared lock would compromise application scalability because Alice reviews the `Post` in the user-think time. For this reason, an optimistic lock should be acquired on the `Post` entity to ensure that the entity state hasn't change since it was first loaded.

```java
entityManager.lock(post, LockModeType.OPTIMISTIC);

LOGGER.info("Alice adds a PostComment to the previous Post entity version");
PostComment comment = new PostComment();
comment.setId(1L);
comment.setReview("Chapter 16 is about Caching.");
comment.setPost(post);
entityManager.persist(comment);
```

`LockModeType.OPTIMISTIC` does not acquire an actual lock right way, but instead it schedules a version check towards the end of the currently running transaction.

When executing the test case above while also acquiring the `LockModeType.OPTIMISTIC` on the `Post` entity, Hibernate generates the following output:

```
-- Alice adds a PostComment review to the previous Post entity version
INSERT INTO post_comment (post_id, review, version, id)
VALUES (1, 'Chapter 16 is about Caching.', 0, 1)

SELECT version FROM post WHERE id = 1

javax.persistence.OptimisticLockException: Newer version [1] of entity
[[Post#1]] found in database
```

`LockModeType.OPTIMISTIC` instructs Hibernate to check the `Post` entity version towards the end of the transaction. If the version has changed, an `OptimisticLockException` is thrown.

16.2.2.1 Inconsistency risk

Unfortunately, this kind of application-level check is always prone to inconsistencies due to bad timing. For example, after Hibernate executes the version check select statement, a concurrent transaction can simply update the `Post` entity without the first transaction noticing anything.

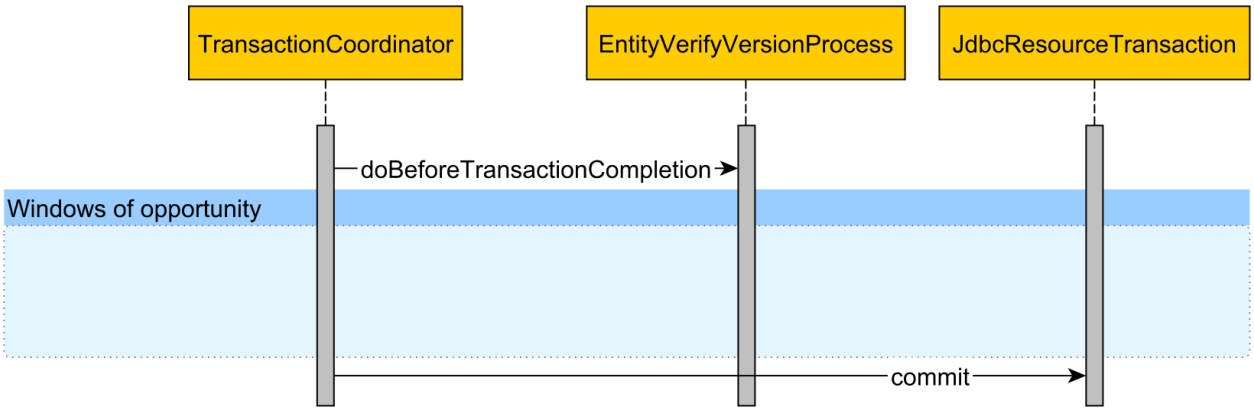

Figure 16.9: LockModeType.OPTIMISTIC window of opportunity

During that *window of opportunity*, another concurrent transaction might change the `Post` entity record before the first transaction commits its changes. To prevent such an incident, the
`LockModeType.OPTIMISTIC` should be accompanied by a shared lock acquisition:

```
entityManager.lock(post, LockModeType.OPTIMISTIC);
entityManager.lock(post, LockModeType.PESSIMISTIC_READ);
```

This way, no other concurrent transaction can change the `Post` entity until the current transaction is ended.

16.2.3 LockModeType.OPTIMISTIC_FORCE_INCREMENT

`LockModeType.OPTIMISTIC_FORCE_INCREMENT` allows incrementing the locked entity version even if the entity hasn't changed at all in the currently running Persistence Context.

The `@Version` attribute should never have a setter method because this attribute is managed automatically by Hibernate. To increment the version of a given entity, one of the two `FORCE_INCREMENT` lock strategies must be used instead.

To understand how the `LockModeType.OPTIMISTIC_FORCE_INCREMENT` strategy works, consider the following Version Control system:

Figure 16.10: Repository, Commit, and Change

The `Repository` is the root entity, and each change is represented by a `Commit` entry which, in turn, may contain one or more `Change` embeddable types.

In this particular example, the `Repository` version must be incremented with each new `Commit` being added. The `Repository` entity version is used to ensure that commits are applied sequentially, and a user is notified if a newer commit was added since she has updated her working copy.

The following example depicts the user flow for this particular Version Control system:

```
Repository repository = entityManager.find(Repository.class, 1L,
        LockModeType.OPTIMISTIC_FORCE_INCREMENT);

Commit commit = new Commit(repository);
commit.getChanges().add(new Change("FrontMatter.md", "0a1,5..."));
commit.getChanges().add(new Change("HibernateIntro.md", "17c17..."));

entityManager.persist(commit);
```

When Alice executes the commit command, every file that she changes is going to be represented by a `Change` embeddable which also holds the diff between the original and the

current file content. The `Repository` entity is loaded using the `LockModeType.OPTIMISTIC_FORCE_INCREMENT` lock strategy so that its version is going to be incremented at the end of the current transaction.

Upon running the aforementioned test case, Hibernate generates the following statements:

```
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r WHERE  r.id = 1

INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,5...', 'FrontMatter.md')
INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '17c17...', 'HibernateIntro.md')

UPDATE repository SET version = 1 WHERE id = 1 AND version = 0
```

The `Repository` version is incremented before transaction completion, and, unlike `LockModeType.OPTIMISTIC`, data integrity is guaranteed by the current transaction isolation level. If the `Repository` version changed in between, the update will fail and an `OptimisticLockingException` is going to trigger a transaction rollback.

In the following example, both Alice and Bob are going to issue two commits concurrently:

```
doInJPA(entityManager -> {
    Repository repository = entityManager.find(Repository.class, 1L,
        LockModeType.OPTIMISTIC_FORCE_INCREMENT);

    executeSync(() -> {
        doInJPA(_entityManager -> {
            Repository _repository = _entityManager.find(Repository.class, 1L,
                LockModeType.OPTIMISTIC_FORCE_INCREMENT);

            Commit _commit = new Commit(_repository);
            _commit.getChanges().add(new Change("Intro.md", "0a1,2..."));
            _entityManager.persist(_commit);
        });
    });

    Commit commit = new Commit(repository);
    commit.getChanges().add(new Change("FrontMatter.md", "0a1,5..."));
    commit.getChanges().add(new Change("HibernateIntro.md", "17c17..."));
    entityManager.persist(commit);
});
```

When running the aforementioned test case, Hibernate generates the following output:

```
-- Alice selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1

-- Bob selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1

-- Bob adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,2...', 'Intro.md')

-- Bob increments the Repository version
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

-- Alice adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 3)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (3, '0a1,5...', 'FrontMatter.md')

INSERT INTO commit_change (commit_id, diff, path)
VALUES (3, '17c17...', 'HibernateIntro.md')

-- Alice increments the Repository version
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

--Exception thrown
javax.persistence.RollbackException: Error while committing the transaction
Caused by: javax.persistence.OptimisticLockException:
Caused by: org.hibernate.StaleObjectStateException:
Row was updated or deleted by another transaction
(or unsaved-value mapping was incorrect)
```

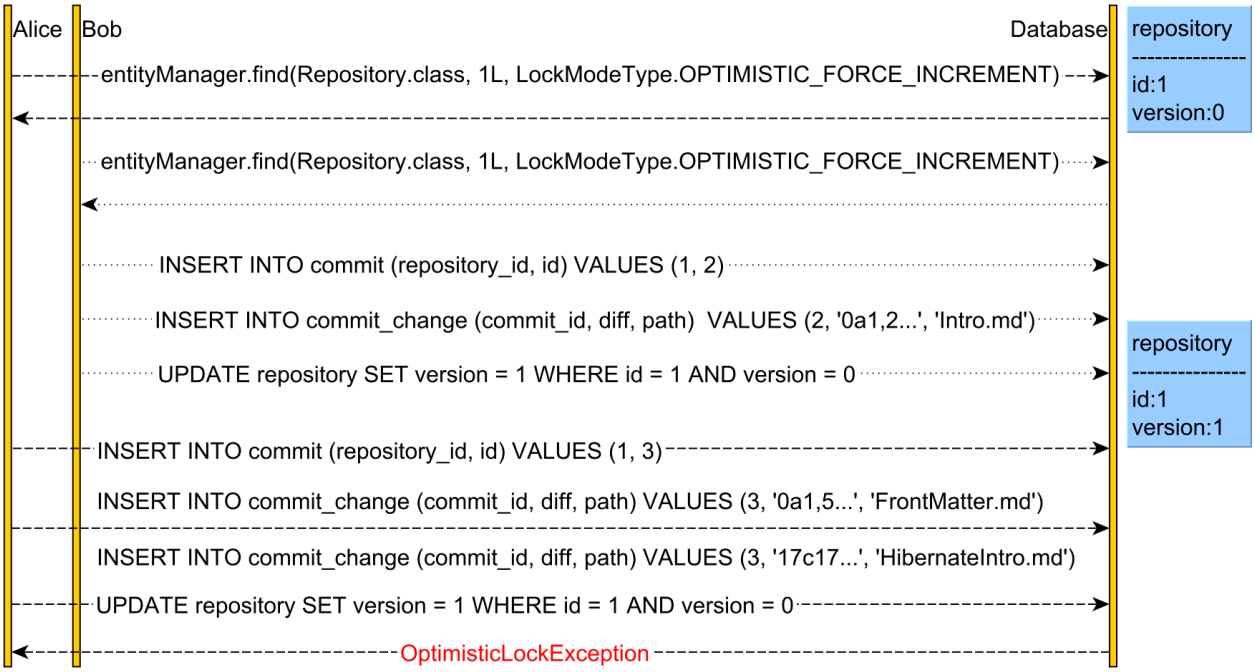

Figure 16.11: LockModeType.OPTIMISTIC_FORCE_INCREMENT

The flow can be explained as follows:

- Alice fetches the `Repository` entity and instructs Hibernate to acquire an `OPTIMISTIC_FORCE_INCREMENT` application-level lock.
- Alice's thread is suspended by the JVM thread scheduler, so Bob gets the chance to fetch the `Repository` entity using the `OPTIMISTIC_FORCE_INCREMENT` lock strategy.
- Bob manages to add a new `Commit` entity, and his transaction is committed. Therefore, the `Repository` entity version is also incremented.
- Alice thread is resumed, and she adds one `Commit` entity and initiates a transaction commit.
- The optimistic locking update fails because the `Repository` version has changed.

`OPTIMISTIC_FORCE_INCREMENT` is useful for propagating a child entity state change to the parent entity optimistic locking version. By applying an optimistic lock on a common parent entity, it is, therefore, possible to coordinate multiple child entities whose changes need to be applied sequentially so that no update is being lost.

16.2.4 LockModeType.PESSIMISTIC_FORCE_INCREMENT

Just like `OPTIMISTIC_FORCE_INCREMENT`, `PESSIMISTIC_FORCE_INCREMENT` can be used to increment the version of any given entity. However, if for `OPTIMISTIC_FORCE_INCREMENT` the entity version is incremented towards the end of the currently running transaction, the `PESSIMISTIC_FORCE_INCREMENT` forces the version incrementation right away, as demonstrated by the test case below.

```
Repository repository = entityManager.find(Repository.class, 1L,
        LockModeType.PESSIMISTIC_FORCE_INCREMENT);

Commit commit = new Commit(repository);
commit.getChanges().add(new Change("FrontMatter.md", "0a1,5..."));
commit.getChanges().add(new Change("HibernateIntro.md", "17c17..."));

entityManager.persist(commit);
```

Hibernate generates the following statements:

```
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1
FOR UPDATE

UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,5...', 'FrontMatter.md')

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '17c17...', 'HibernateIntro.md')
```

The `Repository` is locked using a `FOR UPDATE` SQL clause in the select statement that fetches the aforementioned entity. The `Repository` entity is also incremented before the entity is returned to the data access layer.

In the following example, Alice is going to lock the `Repository` only after she previously fetched the very same entity. However, in the meanwhile, Bob is going to increment the `Repository` entity version using a `PESSIMISTIC_FORCE_INCREMENT` lock.

```
Repository repository = entityManager.find(Repository.class, 1L);

executeSync(() -> {
    doInJPA(_entityManager -> {
        Repository _repository = _entityManager.find(Repository.class, 1L,
            LockModeType.PESSIMISTIC_FORCE_INCREMENT);

        Commit _commit = new Commit(_repository);
        _commit.getChanges().add(new Change("Intro.md", "0a1,2..."));
        _entityManager.persist(_commit);
    });
});

entityManager.lock(repository, LockModeType.PESSIMISTIC_FORCE_INCREMENT);
```

When running the test case above, Hibernate generates the following output:

```
-- Alice selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1

-- Bob selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1
FOR UPDATE

-- Bob increments the Repository version upon fetching the Repository entity
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

-- Bob adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,2...', 'Intro.md')

-- Alice tries to increment the Repository entity version
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

-- Exception thrown
javax.persistence.OptimisticLockException:
Caused by: org.hibernate.StaleObjectStateException:
```

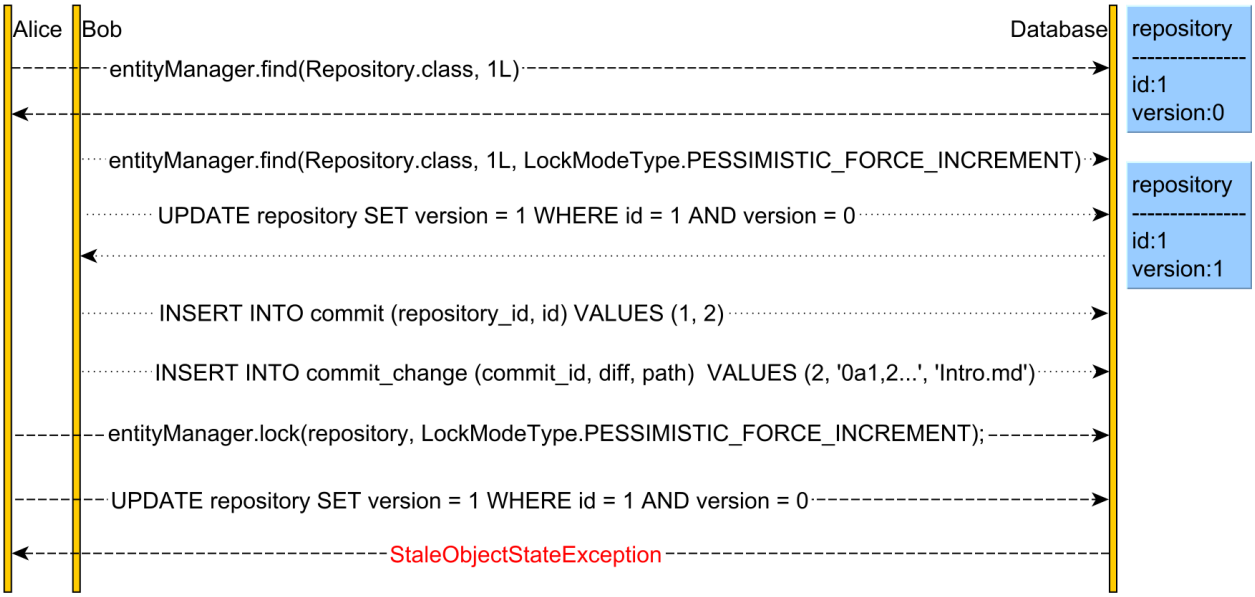

Figure 16.12: Fail fast PESSIMISTIC_FORCE_INCREMENT lock acquisition

The flow can be explained as follows:

- Alice fetches the `Repository` entity without acquiring any physical or logical lock.
- Alice's thread is suspended by the JVM thread scheduler, so Bob gets the chance to fetch the `Repository` entity using the `PESSIMISTIC_FORCE_INCREMENT` lock strategy.
- The `Repository` entity version is incremented right away.
- Bob manages to add a new `Commit` entity, and his transaction is committed.
- Alice thread is resumed, and she attempts to acquire a `PESSIMISTIC_FORCE_INCREMENT` on the already fetched `Repository` entity.
- The optimistic locking update fails because the `Repository` version was changed by Bob.

The instantaneous version incrementation has the following benefits:

- Because the entity is locked at the database row level, the entity version incrementation is guaranteed to succeed.
- If the entity was previously loaded without being locked and the `PESSIMISTIC_FORCE_INCREMENT` version update fails, the currently running transaction can be rolled back right away.

Once a transaction acquires the `PESSIMISTIC_FORCE_INCREMENT` lock and increments the entity version, no other transaction can acquire a `PESSIMISTIC_FORCE_INCREMENT` lock because the second select statement is blocked until the first transaction releases the row-level physical lock.

The following example aims to demonstrate how two concurrent transactions can be coordinated through a common entity `PESSIMISTIC_FORCE_INCREMENT` lock acquisition.

```
doInJPA(entityManager -> {
    Repository repository = entityManager.find(Repository.class, 1L,
        LockModeType.PESSIMISTIC_FORCE_INCREMENT);

    executeAsync(() -> doInJPA(_entityManager -> {
        startLatch.countDown();
        Repository _repository = _entityManager.find(Repository.class, 1L,
            LockModeType.PESSIMISTIC_FORCE_INCREMENT);

        Commit _commit = new Commit(_repository);
        _commit.getChanges().add(new Change("Intro.md", "0a1,2..."));
        _entityManager.persist(_commit);
        _entityManager.flush();
        endLatch.countDown();
    }));
    awaitOnLatch(startLatch);
    LOGGER.info("Sleep for 500ms to delay the other transaction");
    sleep(500);

    Commit commit = new Commit(repository);
    commit.getChanges().add(new Change("FrontMatter.md", "0a1,5..."));
    commit.getChanges().add(new Change("HibernateIntro.md", "17c17..."));

    entityManager.persist(commit);
});
endLatch.await();
```

The `awaitOnLatch` and `sleep` method utilities have the role of converting the `InterruptedException` into a `RuntimeException` which, unlike checked exceptions, can be propagated throughout a lambda expression without having to add unnecessary try/catch clauses.

When executing the test case above, Hibernate generates the following statements:

```sql
-- Alice selects the Repository entity
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1
FOR UPDATE

-- Alice increments the Repository version upon fetching the Repository entity
UPDATE repository SET version = 1 WHERE id = 1 AND version = 0

-- Bob tries to select the Repository entity, but his select is blocked
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r
WHERE  r.id = 1
FOR UPDATE

-- Alice waits 500 ms to delay Bob's lock acquisition request

-- Alice adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,5...', 'FrontMatter.md')

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '17c17...', 'HibernateIntro.md')

-- Bob's increments the Repository version upon fetching the Repository entity
UPDATE repository SET version = 2 WHERE id = 1 AND version = 1

-- Bob adds a new Commit entity
INSERT INTO commit (repository_id, id) VALUES (1, 3)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (3, '0a1,2...', 'Intro.md')
```

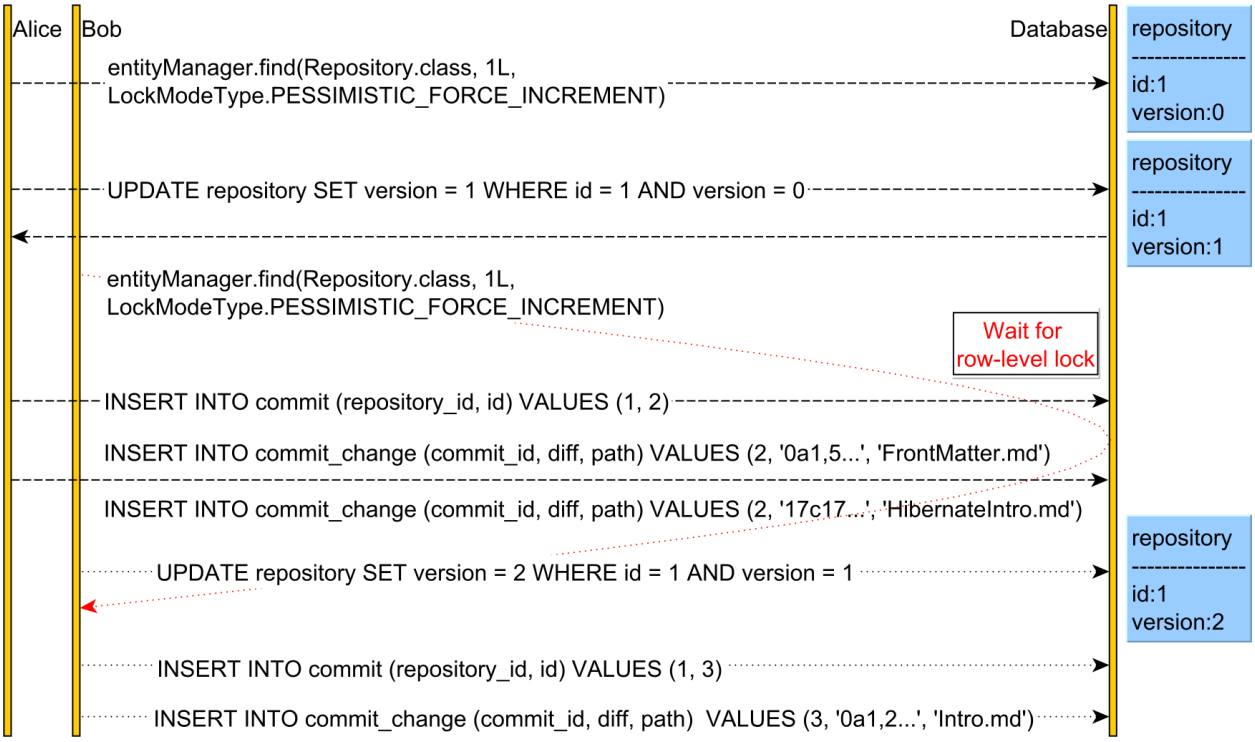

Figure 16.13: LockModeType.PESSIMISTIC_FORCE_INCREMENT

The flow can be explained as follows:

- Alice fetches the `Repository` entity while also acquiring a row-level lock and incrementing the entity version.
- Alice's thread is suspended by the JVM thread scheduler, so Bob gets the chance to fetch the `Repository` entity using the `PESSIMISTIC_FORCE_INCREMENT` lock strategy.
- Because it uses a `FOR UPDATE` clause, Bob `Repository` entity select statement is blocked by Alice's row-level lock.
- Alice thread is resumed, and she waits for 500 ms to delay Bob's lock acquisition request.
- Alice adds a new `Commit` entity, and her transaction is committed. Therefore, the `Repository` entity row-level lock is released.
- Bob can resume his select statement, and he acquires a row-level lock on the `Repository` entity.
- The `Repository` entity version is incremented by the `PESSIMISTIC_FORCE_INCREMENT` lock strategy.
- Bob manages to add his `Commit` entity and commits his transaction.

Once the row-level lock is acquired, the entity version update is guaranteed to succeed, therefore, reducing the likelihood of getting an `OptimisticLockingException`.

III JOOQ

17. Why jOOQ matters

When working with a relational database, it all boils down to SQL statements.

As previously explained, Hibernate entity queries are suitable for read-write logical transactions. For reporting, analytics or ETL (Extract, Transform, Load) native SQL queries are the best choice since they can take advantage of database-specific features like window functions or Common Table Expressions. Even for CRUD operations, there might be times when a database-specific syntax is more suitable like it's the case for the upsert SQL operation[1].

While Hibernate does a very good job to automate the vast majority of statements, it is unlikely that you can rely on Hibernate alone for every business use case. Therefore, native queries are a necessity for most enterprise applications.

As demonstrated in the Native query DTO projection section, both JPA and Hibernate provide a way to execute native SQL statements. Being able to execute any SQL statement is great, but, unfortunately, the JPA approach is limited to static statements only. To build native SQL statement dynamically, JPA and Hibernate are no longer enough.

17.1 How jOOQ works

JOOQ is a query builder framework that allows you generate a great variety of database-specific statements using a Java API. The `DSLContext` is the starting point to building any SQL statement, and it requires two things:

- a reference to a JDBC `Connection`
- a database dialect so that it can translate the Java API query representation into a database-specific SQL query

For instance, when using PostgreSQL 9.5, the `DSLContext` can be constructed as follows:

```
DSLContext sql = DSL.using(connection, SQLDialect.POSTGRES_9_5);
```

17.2 DML statements

With the `DSLContext` in place, it's time to show some simple DML statements like insert, update, delete, as well as a trivial select query. What's worth noticing is that the Java API syntax is almost identical to its SQL counterpart, so most jOOQ queries are self-describing.

[1] https://en.wikipedia.org/wiki/Merge_(SQL)